

Template Matching

In dieser Übung soll ein Template Matching Algorithmus implementiert werden um ein größeres Bild nach einem kleineren Muster (Template) zu durchsuchen.

Dabei wird das Template über das größere Bild geschoben, an jeder Position wird der Abstand zwischen den Pixelwerten des Bildes und jenen des Templates berechnet. In einem zweiten Schritt werden die Positionen mit der höchsten Übereinstimmung gefiltert und in einem Bild markiert.

```
In [1]: # Load Modules
import numpy as np
import scipy
import cv2
import matplotlib.pyplot as plt
import plotly
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import skimage

# Helper Functions
from modules.plotting import displayImagePlotly
```

Bilddateien Laden:

Suchbild: img

Template: img_template das Suchbild enthält exakt/Pixelgenau diese Struktur

Template2: img_template2 dieses Bild ist etwas dunkler. Das Muster ist im Suchbild enthalten, jedoch nicht die exakt Pixelgenaue Struktur.

```
In [2]: # Bilddateien einladen und anzeigen
img = cv2.imread(r'images/img_noise.bmp', cv2.IMREAD_GRAYSCALE)
img_template = cv2.imread(r'images/template.bmp', cv2.IMREAD_GRAYSCALE)
img_template2 = cv2.imread(r'images/template2.bmp', cv2.IMREAD_GRAYSCALE)

displayImagePlotly(img, title = 'Suchbild')
displayImagePlotly(img_template, title = 'Template')
displayImagePlotly(img_template2, title = 'Template 2')
```

Implementierung

Aufgabenstellung:

1. Implementieren Sie eine Funktion die die das Template matching durchführt.

- Berechnen sie hierzu an jeder Position die Summe der quadratischen

Abweichungen zwischen jedem Pixelpaar im Überlappbereich zwischen Template und Bild.

- Speichern Sie diese Abweichung in einer neuen Ergebnis-Matrix ab.
 - Geben Sie diese Ergebnis-Matrix-Bild als Rückgabewert der Funktion zurück.
2. Filtern Sie das Ergebnis Bild des sodass ein Binärbild erzeugt wird, das nur an den Positionen mit bester Übereinstimmung den Pixelwert 255 besitzt und ansonsten den Wert 0.

Implementierung Template Matching

Aufgabe:

Implementieren Sie die Funktion `templateMatching`. Diese Funktion soll das Template schritt für Schritt über das Bild schieben und an jeder Position die Summe der quadratischen Abweichungen zwischen den Pixelwerten des Bildes und jenen des Templates berechnen. Bitte beachten Sie hier das Koordinatensystem. Sowohl das Bild als auch das Template besitzen ein Koordinatensystem, das in der linken oberen Ecke bei Pixel (0,0) beginnt. Da das Template nicht über die Ränder des Bildes hinaus geschoben werden kann, muss die Schleife nur bis zur Position (Bildbreite - Templatebreite, Bildhöhe - Templatehöhe) laufen. Das Ergebnisbild ist dann auch dementsprechend kleiner als das Eingangsbild.

Hinweis

- Die Berechnung des Mittelwertes eines Bildes / Bildausschnitts lässt sich mit der Funktion `np.mean(img)` einfach berechnen <https://numpy.org/devdocs/reference/generated/numpy.mean.html>
- Einfache Rechenoperationen in numpy:
 - `img**2` quadriert jeden Pixel eines Bildes
 - `img-const` subtrahiert den Wert `const` von jedem Pixelwert des Bildes
 - `img1 - img2` subtrahiert den jeweiligen Pixelwert des Bildes `img1` vom Bild `img2`

```
In [3]: def templateMatching(image, template):
        """
        Template Matching implementieren
        Input:
            image: Gray Scale Image (2D numpy array, 8bit)
            template: template (2D numpy array, 8bit)

        Output:
            result: deviation image (2D numpy array, float32)

        Notes:
            - Calculate the mean squared error between the template and the image patch
            - Since the template cannot be applied at the borders of the image, the
              - For example, if the input image is 100x100 and the template is 10x10,
```

```

- You can assume that the template is smaller than the image
"""

#####
## Enter your Code Here
#####
# Größe des Bildes und des Templates bestimmen
img_height, img_width = image.shape
temp_height, temp_width = template.shape

# fuer Berechnungen Bilder als int umwandeln
image = image.astype(int)
template = template.astype(int)

# Ergebnis-Matrix initialisieren
result_height = img_height - temp_height-1
result_width = img_width - temp_width-1
result = np.zeros((result_height, result_width), dtype=int)

# Template Matching durchführen
for y in range(result_height):
    for x in range(result_width):
        # Bildausschnitt extrahieren
        image_patch = image[y:y+temp_height, x:x+temp_width]
        # Mittlere quadratische Abweichung berechnen
        mse = np.mean((image_patch - template) ** 2)
        # Abweichung in der Ergebnis-Matrix speichern
        result[y, x] = mse

return result

```

Wenden Sie nun die Funktion `templateMatching` auf das Bild `image` und das Template `template` an und speichern Sie das Ergebnis-Bild in der Variable `match`. Filtern Sie anschließend die besten Übereinstimmungen mit einem geeigneten Schwellwert und speichern Sie die binäre Maske in der Variable `bestMatches` mit Datentyp `uint8`. (0: keine Übereinstimmung, 255: beste Übereinstimmung)

Hinweis / Schwellwertbildung

der Code: `filtered = image < 10` macht eine Schwellwertbildung des Bildes `image`. Danach erhält man ein Bild aus 0 und 1. Ist das Bild 1 ist die Bedingung wahr.

In [4]: `# Template Matching aufrufen`

```

#####
## ENTER YOUR CODE HERE
#####
match = templateMatching(img, img_template)

# Das Ergebnis ist ein Float32-Bild: das als heatmap () dargestellt werden kann
fig = px.imshow(match, color_continuous_scale='viridis',
                  title='Template Matching Ergebnis (Mean Squared Error)')

```

```
fig.show()
```

Template Matching Ergebnis (Mean Squared Error)



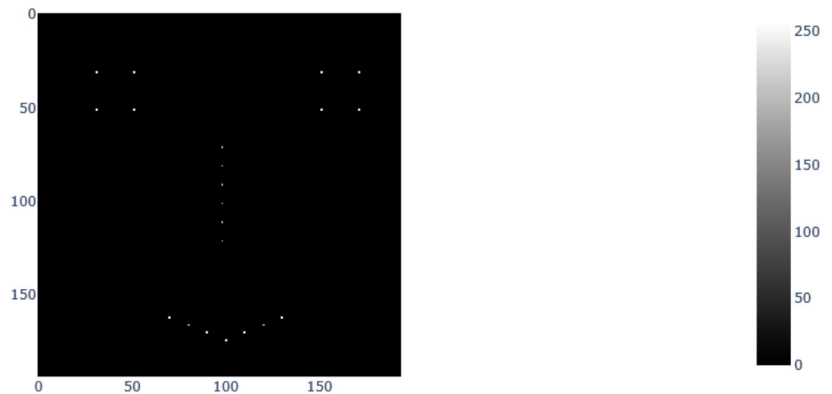
```
In [5]: # Beste Übereinstimmungen finden und als Binärbild darstellen

#####
## ENTER YOUR CODE HERE
#####

bestMatches = match < 10
bestMatches = bestMatches.astype(np.uint8) * 255

# Hinweis: Skalieren Sie das Bild vor der Darstellung ggf. auf den Wertebereich 0-255
displayImagePlotly(bestMatches, title = 'Ergebnis Template Matching')
```

Ergebnis Template Matching



Geben Sie nun noch die Positionen der besten Übereinstimmungen aus. Nutzen Sie hierzu die Funktion `np.where`

```
In [6]: ###
## ENTER YOUR CODE HERE
###

matches = np.where(bestMatches == 255)
print(f'y-Koordinaten der Matches: {matches[0]}')
print(f'x-Koordinaten der Matches: {matches[1]}')
```

```

y-Koordinaten der Matches: [ 31  31  31  31  51  51  51  51  71  81  91 101 111 121
162 162 166 166
 170 170 174]
x-Koordinaten der Matches: [ 31  51 151 171  31  51 151 171  98  98  98  98  98  98
 70 130  80 120
  90 110 100]

```

Test mit dem etwas dunklerem Template

Mit dem etwas dunkleren Template `img_template2` funktioniert dieses hier vorgestellte template matching nicht mehr. Die Struktur ist nicht mehr exakt Pixelgleich im Suchbild enthalten. Führen Sie nun Ihren template matching algorithmus mit diesem Bild aus. Was beobachten Sie.

```

In [7]: match2 = templateMatching(img, img_template2)

# Das Ergebnis ist ein Float32-Bild: das als heatmap () dargestellt werden kann
fig = px.imshow(match2, color_continuous_scale='viridis',
                 title='Template Matching Ergebnis (Mean Squared Error)')

fig.show()

bestMatches2 = match2 < 10
bestMatches2 = bestMatches2.astype(np.uint8) * 255

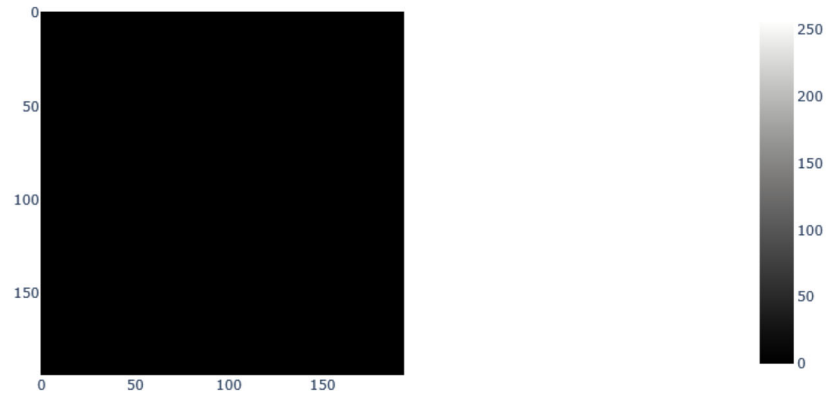
# Hinweis: Skalieren Sie das Bild vor der Darstellung ggf. auf den Wertebereich 0-2
displayImagePlotly(bestMatches2, title = 'Ergebnis Template Matching')

```

Template Matching Ergebnis (Mean Squared Error)



Ergebnis Template Matching



Template Matching Mittels Kreuzkorrelation

In der Python Funktion `skimage.feature.match_template()` ist die robustere Variante eines template matching Algorithmus implementiert. Diese ist gegen Rauschen und gegen Helligkeitsunterschiede nicht sensitiv.

Aufgabe

Wenden Sie die Funktion auf das Testbild und das `img_match2` mit veränderter Helligkeit an. Beachten Sie, dass im Falle einer größtmöglichen Übereinstimmung die Korrelation = 1 ist. Um die Position der besten Übereinstimmung zu benennen, müssen daher auch die Filterbedingung, mit der höchste Übereinstimmung identifiziert werden kann angepasst werden.

```
In [8]: # Template Matching algorithmus anwenden und Korrelationsbilde anzeigen.

#####
## ENTER YOUR CODE HERE
#####
correlation = skimage.feature.match_template(img, img_template2)

# Das Ergebnis ist ein Float32-Bild: das als heatmap () dargestellt werden kann
fig = px.imshow(correlation, color_continuous_scale='viridis',
                 title='Template Matching Ergebnis (Mean Squared Error)')
fig.show()
```

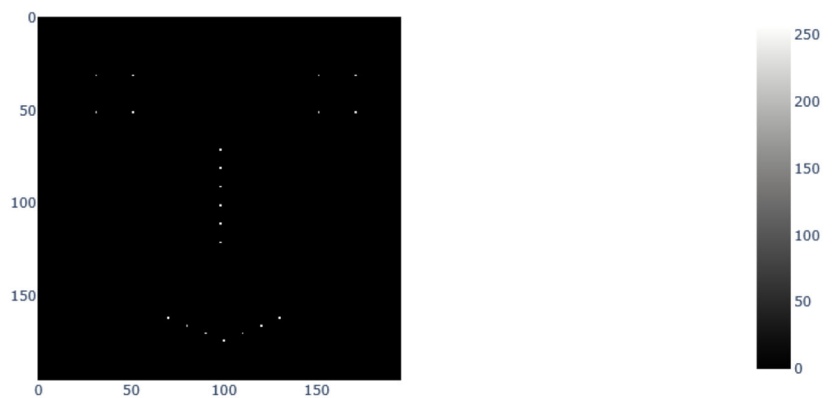
Template Matching Ergebnis (Mean Squared Error)



```
In [9]: # Beste übereinstimmungen filtern und als Binärbild darstellen
#####
## ENTER YOUR CODE HERE
#####
filtered_correlation = correlation > 0.8
filtered_correlation = filtered_correlation.astype(np.uint8) * 255

displayImagePlotly(filtered_correlation, title = 'Beste übereinstimmungen')
```

Beste übereinstimmungen



Freiwillige Zusatzaufgabe

modifizieren Sie die von Ihnen implementierte `templateMatching` Funktion, sodass diese das Matching über die Berechnung des Korrelationskoeffizienten durchführt.

```
In [10]: # def templateMatching_Correlation(image, template):
```